SIMATS ENGINEERING

Reverse Engineering Task

COURSE NAME: Software Testing **COURSE CODE: CSA3715**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A partially tested ATM withdrawal module is provided where only input-output behavior is known (e.g., withdrawal limits, balance validation, error messages). Reverse engineer the system by analyzing the observed behavior and reconstructing the underlying test case design strategy. Decompose the system into functional components and identify possible equivalence classes and boundary conditions. Infer the black box testing techniques used and evaluate their adequacy. Reconstruct an improved set of test cases covering all critical scenarios. Document your analysis with structured explanation.	5
2	A software module's execution traces and code snippets are given, but no test documentation is available. Reverse engineer the white box testing approach by constructing the control flow graph from the given logic. Identify all decision points, loops, and independent execution paths. Evaluate the existing test adequacy in terms of coverage criteria. Reconstruct a complete path testing strategy ensuring full coverage. Present your findings with clear reasoning and structured representation.	5
3	A web application login and transaction system has been tested using unknown techniques, but only defect logs and test results are available. Reverse engineer the testing approach by analyzing defect patterns and identifying which black box and white box techniques were likely used. Decompose the system into testable components and evaluate gaps in testing coverage. Reconstruct a comprehensive testing strategy integrating equivalence partitioning, boundary value analysis, and path testing. Justify your reconstructed approach with logical reasoning and structured documentation.	5

4	A legacy inventory management system provides only a set of successful and failed transaction logs regarding stock updates and low-stock alerts. Reverse engineer the business logic by analyzing these logs to identify the hidden boundary conditions and equivalence partitions. Decompose the update process into functional units and reconstruct a black box test design that addresses the gaps found in the logs. Justify your selection of test cases based on the inferred logic.	5
5	You are provided with a compiled .exe file of a library management module and its corresponding CPU/Memory execution profile during a stress test. Reverse engineer the white box testing requirements by mapping execution spikes to potential loops and decision clusters in the logic. Construct a conceptual control flow graph (CFG) that represents this behavior. Reconstruct a path testing strategy that targets these resource-intensive paths and evaluate its impact on system stability.	6
6	A mobile payment gateway's API responses (JSON) for various transaction states—Pending, Success, Reversal, and Timeout—are given. Reverse engineer the underlying state machine by analyzing the transition triggers and data constraints. Identify the implicit state transition testing technique used. Reconstruct a comprehensive state-based test suite that ensures all legal and illegal transitions are covered. Document your analysis with a structured state-transition diagram.	5
7	An encrypted communication module provides only "black-box" behavior through its input parameters and encrypted outputs. Reverse engineer the validation logic by performing "fuzzing" analysis on the input fields to identify error-handling boundaries. Decompose the module into data validation and encryption components. Reconstruct an improved set of test cases using Boundary Value Analysis (BVA) and justify how your design prevents buffer overflow or injection vulnerabilities.	5
8	A set of legacy unit test results for an insurance premium calculator shows 100% statement coverage but fails to catch defects in multi-condition "if" statements. Reverse engineer the original white box approach and identify the inadequacy in the coverage criteria used. Construct a new CFG and identify all independent execution paths. Reconstruct a path testing strategy that achieves 100% Branch and Path coverage. Explain how this reconstruction reduces the risk of calculation errors.	5

9	A cloud-based file synchronization tool provides user activity logs and a list of "lost file" defect reports. Reverse engineer the synchronization algorithm's logic by analyzing the defect patterns related to file size, type, and network speed. Infer the likely Equivalence Class Partitioning (ECP) used by the original developers. Reconstruct a revised testing strategy that integrates BVA with stress testing. Justify how your reconstructed approach prevents future data loss.	5
10	You are given a sequence of database queries (SQL) generated by an e-commerce "Search and Filter" module, but no source code. Reverse engineer the search logic by analyzing the query parameters and the resulting dataset. Identify the decision-making logic used for complex filtering (e.g., AND/OR/NOT). Reconstruct a Decision Table based testing strategy that covers all possible search combinations. Evaluate the completeness of your reconstructed model.	5
11	A smart thermostat's firmware execution trace shows a recurring pattern of "nested loops" during temperature calibration. Reverse engineer the white box testing strategy by identifying the loop invariants and termination conditions from the trace. Decompose the calibration module into its logical sub-routines. Reconstruct a path testing strategy specifically targeting the boundary values of the loops. Justify your approach in terms of preventing firmware hangs or infinite loops.	5
12	A third-party "Tax Calculation" DLL provides specific error codes for different income brackets but has no documentation. Reverse engineer the tax-slab logic by analyzing the output error codes against varied input ranges. Decompose the system into functional components based on income categories. Reconstruct a set of test cases using ECP and BVA. Provide a structured explanation of how your reconstructed test suite validates the correctness of the hidden tax laws.	5
13	An automated flight booking system's integration test reports show that errors only occur when "Frequent Flyer" status is combined with "International" travel and "Last-minute" booking. Reverse engineer the testing approach to identify why these multi-variable combinations were missed. Apply a combination of black box (Cause-Effect Graphing) and white box (Path Testing) to reconstruct a hybrid testing strategy. Justify how this approach ensures maximum defect detection in complex business workflows.	5

Code of Conduct:

*I **B.Hari Sai/192311098** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:B.Hari Sai

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

ANSWERS:

Q1. Reverse Engineering an ATM Withdrawal Module

The ATM withdrawal module can be treated as a black box because its internal code is unavailable. Its behavior can be inferred by supplying controlled inputs and comparing the observed messages and account balance with expected results.

Functional components

- Input validation: checks whether the amount is numeric, positive, and in an accepted denomination.
- Withdrawal-limit validation: checks per-transaction and daily limits.
- Balance validation: verifies that the account is active and has sufficient available funds.
- Transaction processing: debits the account only after all validations pass.
- Response handling: returns success, receipt details, or a meaningful error without changing the balance on failure.

Equivalence classes and boundaries

Input / rule	Equivalence classes	Boundary tests
Withdrawal amount	Invalid: text, blank, negative, zero; valid: positive permitted amount; invalid: unsupported denomination	-1, 0, minimum permitted, minimum−one unit, maximum permitted, maximum+one unit
Available balance	Insufficient, exact balance, sufficient balance	Balance−one unit, exact balance, balance+one unit
Limits	Below limit, at limit, above limit	Limit−one unit, limit, limit+one unit
Account/card status	Active, blocked, expired, unknown account	One test for each state

Likely technique and adequacy

The observed examples suggest equivalence partitioning, boundary value analysis (BVA), and error guessing. These are suitable black-box techniques, but a few ordinary successful withdrawals are not enough. Adequacy requires testing every meaningful class, values immediately around limits, repeated withdrawals near the daily limit, and the rule that failed transactions must not debit the account.

Improved test cases

ID	Test input / condition	Expected result
ATM-01	Valid amount below all limits; sufficient balance	Approved; balance decreases by exact amount
ATM-02	Amount is blank, text, or non-numeric	Validation error; balance unchanged
ATM-03	Amount is negative or zero	Rejected; balance unchanged
ATM-04	Amount is one unit below minimum	Rejected if below configured minimum
ATM-05	Amount equals minimum and maximum allowed values	Accepted if funds and other rules permit
ATM-06	Amount is one unit above transaction limit	Limit error; no debit
ATM-07	Amount exceeds available balance by one unit	Insufficient-funds error; no debit
ATM-08	Amount equals available balance	Approved only if account and limit rules allow it
ATM-09	Daily total reaches limit exactly, then another withdrawal is attempted	First transaction follows policy; next is rejected if it exceeds the limit
ATM-10	Blocked/expired card or inactive account	Appropriate error; no debit
ATM-11	Network timeout after request submission	Transaction status is reconciled; no duplicate debit on retry
ATM-12	Cash dispenser failure after authorization	Reversal/rollback occurs according to system policy

Conclusion: combine ECP and BVA with state and transaction-failure tests. Verify both the returned response and the resulting account balance.

Q2. Reconstructing White-Box Testing from Code and Execution Traces

White-box reverse engineering starts by converting the available logic into a control flow graph (CFG). Each basic block becomes a node, each decision creates alternative edges, and loops create back edges. Since the actual snippets are not included in the question,

the following is a method and illustrative model rather than a claim about a specific unseen program.

CFG construction procedure

- List sequential statements and group statements with one entry and one exit into basic blocks.
- Create a decision node for every if/else, switch case, compound condition, and loop condition.
- Connect nodes in execution order; add true and false edges for decisions and a back edge from each loop body to its condition.
- Add explicit entry, normal exit, and exception/error exit nodes where applicable.

Coverage and independent paths

Decision points, loop conditions, early returns, exception handlers, and short-circuit Boolean expressions must be identified. Cyclomatic complexity gives a lower bound on the number of linearly independent paths: $V(G) = E - N + 2P$, where E is the number of edges, N the number of nodes, and P the number of connected components (normally 1).

Coverage criterion	What it proves	Limitation
Statement coverage	Every executable statement ran at least once	May miss an untested decision outcome
Branch/decision coverage	Every decision outcome (true/false or each case) ran	May not cover all combinations of conditions
Condition coverage	Each atomic Boolean condition was true and false	Does not necessarily prove every decision outcome
Modified Condition/Decision Coverage (MC/DC)	Each condition independently affects the decision outcome	More rigorous for safety-critical logic
Basis path testing	A basis set of independent paths is executed	All possible paths may be infinite when loops are unbounded

Reconstructed path-testing strategy

- Create a node/edge table and label each decision outcome.
- Calculate V(G), then select at least V(G) independent paths for basis-path coverage.
- Design inputs to force each branch and each early exit, including invalid data and exception paths.
- For each loop, test zero iterations, one iteration, a typical count, the maximum permitted count, and a value that triggers termination.

- Measure statement, branch, condition, and path coverage; document unreachable code and justify any infeasible paths.

Conclusion: traces alone can indicate executed behavior, but they do not prove unexecuted paths are correct. The reconstructed CFG and coverage report make the missing paths explicit.

Q3. Login and Transaction System: Hybrid Black-Box and White-Box Strategy

Defect logs can suggest the techniques previously used, but cannot prove exactly which techniques were applied. For example, defects only at minimum/maximum values suggest missing BVA; defects in combinations of roles, account status, or transaction type suggest weak combination testing; crashes on rare error routes suggest insufficient branch or exception-path testing.

Testable components

- Login and identity validation: username/email, password, account status, lockout, MFA, and session creation.
- Authorization: role, ownership, permissions, and access to protected operations.
- Transaction validation: amount, balance, limits, beneficiary, and transaction status.
- Persistence and integration: database commit/rollback, duplicate request handling, and external service errors.
- Audit and response layer: logs, error codes, session expiry, and sensitive-data handling.

Reconstructed test design

Technique	Application	Example tests
ECP	Partition valid and invalid credentials, account states, transaction amounts, and permissions	Correct credentials; wrong password; disabled account; malformed username
BVA	Test just below, at, and just above limits	Password length min−1/min/min+1; amount limit−1/limit/limit+1; lockout threshold−1/threshold
Decision/combination testing	Test meaningful combinations of authentication and	Valid login + insufficient role; locked account + correct password; MFA

	authorization conditions	failure + correct password
CFG/path testing	Exercise success, rejection, retry, timeout, rollback, and exception routes	Login success; validation failure; DB error; payment timeout; compensating reversal
Security testing	Check injection, brute force, session handling, and authorization bypass	SQL injection strings, rate-limit checks, expired session, direct API call without permission

Adequacy evaluation

A strong suite should report statement and branch coverage for code, boundary and equivalence-class coverage for inputs, and transaction-state coverage for the workflow. Include negative tests, concurrency tests, and checks that failed or retried requests do not create duplicate transactions. Defect logs should be mapped to missing test conditions and then converted into regression tests.

Conclusion: use black-box tests to validate externally visible requirements and white-box tests to target internal branches. Neither alone is sufficient for a login and transaction system.

Q4. Inventory System: Reconstructing Stock Logic from Logs

Stock logs can reveal whether the system permits zero stock, rejects negative stock, triggers alerts at a threshold, and handles repeated updates consistently. Successful and failed examples should be grouped by initial stock, quantity change, resulting stock, product status, and alert threshold.

Functional units

- Receive stock: add a positive received quantity and record supplier/reference details.
- Issue or sell stock: subtract a positive quantity while enforcing availability rules.
- Validate quantities: reject zero, negative, fractional, or malformed values when not permitted.
- Calculate remaining stock: ensure updates are atomic and avoid lost updates during concurrent transactions.
- Low-stock alert: trigger when stock crosses or equals a configured threshold, according to the stated business rule.

Equivalence classes and boundaries

Variable	Classes	Boundary-focused cases
Update quantity	Positive valid; zero; negative; malformed	−1, 0, 1; maximum allowed quantity and maximum+1
Current stock	Zero, low, normal, maximum capacity	0, 1, threshold−1, threshold, threshold+1, maximum
Resulting stock	Negative, zero, positive, above capacity	Issue exact stock; issue one more than stock; receive to capacity and exceed it
Alert status	Above threshold, at threshold, below threshold	Move stock from threshold+1 to threshold; from threshold to threshold−1; replenish above threshold

Improved test cases

ID	Scenario	Expected result
INV-01	Receive 10 units when stock is 5	Stock becomes 15; one update record is stored
INV-02	Issue fewer units than current stock	Stock decreases by the requested amount
INV-03	Issue exactly the current stock	Stock becomes zero if zero stock is permitted
INV-04	Issue one unit more than available stock	Reject without changing stock
INV-05	Submit zero or negative update quantity	Reject with validation message
INV-06	Stock becomes exactly the low-stock threshold	Alert follows the documented inclusive/exclusive rule
INV-07	Stock crosses from above threshold to below it	Alert is generated once and associated with the correct item
INV-08	Receive stock after an alert	Stock is updated and alert is cleared/resolved according to policy
INV-09	Two users update the same item simultaneously	No lost update; final stock reflects both valid transactions

INV-10	Database failure during update	Transaction rolls back; stock and audit log remain consistent

Conclusion: logs alone cannot establish undocumented business rules with certainty. Confirm the threshold rule with stakeholders, then use ECP, BVA, transition tests, and concurrency tests.

Q5. Library Module: Resource Profiles and Conceptual CFG

A compiled executable and CPU/memory profiles do not reveal the exact source-level CFG. Execution spikes identify suspicious regions, not definitive loops or decisions. To reconstruct a conceptual model, correlate input sizes and execution phases with repeated calls, profile samples, memory allocation patterns, and any available symbols or disassembly.

Conceptual control-flow model

Entry → validate request → load/search records → [records remain?] process record and update result → repeat search loop → format response → exit. Additional branches may include invalid input, missing record, allocation failure, file/database error, and recovery/cleanup. Nested loops may arise when each book is compared with each user request or when each record is scanned for matching fields.

Interpreting resource spikes

- CPU spike that grows with record count may indicate repeated scans, nested loops, sorting, or expensive comparisons.
- Memory growth that does not fall after each request may indicate retained objects, unbounded caches, or a leak.
- A sharp spike at a particular input size may indicate a threshold, resizing, sorting transition, or pathological input.
- Confirm hypotheses by repeating profiles with controlled input sizes and comparing runtime and peak memory; do not infer a specific loop from a spike alone.

Resource-focused path tests

Test	Input condition	Purpose / expected result
LIB-01	Empty library / zero records	Loop executes zero times; normal empty result
LIB-02	One record	Tests the single-iteration

		path
LIB-03	Typical dataset	Baseline latency and memory
LIB-04	Large dataset near supported capacity	Tests repeated and nested loops under load
LIB-05	Maximum permitted request size and one above	Checks bounds and graceful rejection
LIB-06	Many records with no match and with match at first/last position	Exercises worst-case and early-exit search paths
LIB-07	Allocation or storage failure where safely injectable	Cleanup path runs; process remains consistent
LIB-08	Repeated requests over a sustained period	Detects leaks, unbounded growth, and degradation

Evaluate with throughput, latency percentiles, peak memory, CPU utilization, error rate, and process stability. A test suite should include normal, worst-case, and failure paths. Use safe profiling and controlled test environments; resource profiles are evidence for investigation, not a substitute for source-level coverage.

Q6. Mobile Payment Gateway: State-Transition Testing

The JSON responses expose a transaction state machine. A typical model has Pending as an intermediate state, Success as a completed state, Reversal as a compensating outcome, and Timeout as an uncertain/failed-to-confirm outcome. Exact legal transitions depend on the gateway contract and must be verified against its specification.

Conceptual state model

Current state	Trigger/event	Next state	Rule
Pending	Provider confirms payment	Success	Legal if transaction reference and amount match
Pending	Provider declines or settlement fails	Reversal / Failed	Follow API contract; reversal may be a separate compensating state
Pending	Response deadline expires	Timeout	Timeout does not necessarily prove

			payment failed
Timeout	Late success callback arrives	Success or reconciliation state	Must reconcile idempotently according to contract
Success	Duplicate success callback	Success (no-op)	No second debit or duplicate fulfillment
Success	Authorized refund/reversal event	Reversal / Refunded	Only if policy permits and event is authenticated
Reversal	Duplicate reversal event	Reversal (no-op)	Idempotent; no double credit
Reversal	Ordinary success callback arrives late	Remain Reversal or reconcile	Reject stale event unless contract explicitly allows a corrected transition

State-transition diagram in text

Start → Pending → (confirmation) Success. Pending → (decline/compensation) Reversal. Pending → (deadline) Timeout. Timeout → (late callback/reconciliation) Success or Reversal according to verified provider rules. Success and Reversal are terminal for ordinary callbacks, while authorized refund/reconciliation events may allow a documented transition.

Test suite

- Test every legal transition at least once and every prohibited transition with a clear expected error or no-op.
- Check missing, malformed, duplicated, or mismatched transaction IDs, amount, currency, signature, and timestamp in JSON.
- Send duplicate callbacks and callbacks in reversed order; verify idempotency and consistent final state.
- Simulate network timeout before and after provider processing; query status before retrying to avoid duplicate payment.
- Verify state persistence, audit logs, response codes, and reconciliation for uncertain outcomes.

The likely technique is state-transition testing, supported by transition-pair coverage and API contract testing. Transition coverage is adequate only when all meaningful state/event pairs—including invalid and duplicate events—are exercised.

Q7. Encrypted Communication Module: Fuzzing and Boundary Analysis

Treat the module as two logical components: input validation/normalization and encryption/output encoding. Fuzzing supplies malformed, unexpected, and extreme inputs to discover crashes, hangs, inconsistent errors, and unsafe handling. Boundary Value Analysis then targets lengths and numeric limits around known constraints.

Input partitions and boundaries

Field	Partitions	Boundary tests
Message length	Empty, valid length, too long	0, 1, max−1, max, max+1 bytes
Key/nonce/IV length	Supported format/length; too short; too long; malformed encoding	Each required length−1, exact length, length+1
Character encoding	Valid UTF-8; invalid byte sequences; embedded NUL; control characters	Truncated multi-byte sequence; NUL at start/middle/end
Metadata / numeric fields	Valid range; negative; overflow; malformed text	Minimum−1, minimum, maximum, maximum+1; very large integer
Ciphertext / tag	Valid, truncated, altered, wrong key, wrong tag	Empty, one byte short, exact size, extra byte

Reconstructed tests and security controls

- Send random bytes, repeated delimiters, long strings, Unicode edge cases, null values, and malformed JSON; expect controlled validation errors.
- Test oversized messages and metadata to confirm limits are enforced before allocation or copying.
- Verify output length and encoding for valid inputs; verify invalid authentication tags are rejected without exposing plaintext.
- Use known-answer cryptographic test vectors and negative tests for wrong keys, nonce reuse detection where applicable, and tampered ciphertext.
- Run sanitizers, memory-safety checks, and timeouts in a controlled test environment; fuzzing complements rather than replaces security review.

BVA helps detect off-by-one errors and length-handling defects; fuzzing explores unexpected combinations. Preventing buffer overflow also requires safe memory APIs, explicit length checks, bounded allocations, and correct integer-overflow handling.

Injection risk is reduced by validating structured fields and avoiding unsafe command/string construction; encryption alone does not prevent injection.

Q8. Insurance Premium Calculator: Beyond Statement Coverage

100% statement coverage means every executable statement ran, but it does not guarantee that every branch or Boolean condition was evaluated both ways. For example, a test may execute the premium calculation statement while only testing one outcome of a compound condition such as age eligibility AND policy type.

Illustrative CFG

Entry → validate inputs? [No → validation error/exit; Yes → evaluate eligibility] → eligible? [No → reject/exit; Yes → choose risk category] → apply base premium → apply discount/surcharge conditions → return premium. Each decision has true and false edges; compound conditions should be decomposed into their atomic conditions.

Coverage gap and independent paths

- Statement coverage can miss false outcomes, boundary values, and interactions between conditions.
- Branch coverage requires both outcomes of each decision; condition coverage requires each atomic condition to be true and false.
- MC/DC is recommended when each condition must be shown to independently affect a decision.
- A numeric count of independent paths cannot be determined without the actual code. Build the CFG and calculate $V(G) = E - N + 2P$; use this as the basis-path target.

Reconstructed strategy

Test focus	Example input design	Expected check
Input validation	Negative age/income, missing policy type, unsupported values	Reject safely; no premium calculated
Eligibility branch	One eligible and one ineligible customer	Both outcomes and correct messages
Compound decision	Vary one condition at a time, holding others fixed	Demonstrate each condition can change the decision
Risk bracket boundaries	Just below, at, and just above each age/risk/income cutoff	Correct bracket and premium formula
Discount/surcharge	No discount, discount only,	Correct order and

	surcharge only, both if permitted	arithmetic
Rounding and extremes	Very small premium, maximum supported values, fractional values	Defined rounding and no overflow

Branch coverage is measurable and useful, but “100% path coverage” is generally impossible when loops exist or the number of paths is unbounded. The practical target is 100% branch coverage plus basis-path coverage, MC/DC where warranted, and boundary-focused calculation tests. Recalculate expected premiums independently to detect arithmetic and rule-order errors.

Q9. Cloud File Synchronization: Lost Files, ECP, BVA and Stress Testing

Lost-file defects should be grouped by file size, file type, network speed, interruption point, concurrent edits, and sync direction. If defects cluster at large files or slow connections, the original equivalence partitions may have been too broad—for example, treating all “valid files” or all “networks” as one class.

Revised partitions

Dimension	Equivalence classes / boundaries
File size	Zero-byte; small; typical; near maximum; over maximum; exact chunk-size boundaries
File type	Supported common types; supported but unusual names/extensions; unsupported type; Unicode or long filename
Network	Fast/stable; slow/stable; intermittent; offline; high latency; connection drops during upload/download
File state	New file; modified file; deleted file; renamed file; same file edited on two devices; conflict
Storage	Available space; near full; full; permission denied; quota exceeded

Improved test strategy

- Use BVA at maximum file size, chunk boundaries, filename length, quota, and retry-count limits.
- Interrupt the connection before transfer, mid-chunk, after upload but before acknowledgment, and during final metadata commit.
- Run stress tests with many files, many devices, concurrent edits, and long-duration sync; monitor memory, queues, and retry behavior.
- Verify checksums, file counts, version IDs, and metadata before and after sync; confirm that a retry is idempotent.
- Test conflict resolution, deletion propagation, rename operations, duplicate names, clock skew, and out-of-order events.
- Check that incomplete uploads remain recoverable and that the system never marks a file synchronized before durable storage is confirmed.

The inferred ECP should be revised into classes that represent materially different handling paths. BVA covers limits, while stress and fault-injection tests reveal race conditions and recovery defects. Data integrity assertions—hashes, version history, and reconciliation—are essential to demonstrating that lost files are prevented.

Q10. E-commerce Search and Filter: Decision-Table Testing

SQL queries and resulting datasets can reveal the filter semantics. For each parameter, determine whether it is optional, how missing values are treated, whether multiple values use AND or OR, how NOT handles nulls, and whether filters are applied before or after sorting/pagination. Do not infer semantics from one result alone; use controlled records that differ in exactly one attribute.

Example conditions and actions

Condition	Meaning
C1: Category selected?	Yes / No
C2: Price within selected range?	Yes / No
C3: In stock only?	Yes / No
C4: Search keyword matches?	Yes / No

Rule	C1	C2	C3	C4	Expected action
R1	No	Yes	No	Yes	Return keyword

					matches in price range, regardless of category or stock
R2	Yes	Yes	Yes	Yes	Return records matching category AND price AND stock AND keyword
R3	Yes	No	Yes	Yes	Exclude products outside price range
R4	No	Yes	Yes	No	Return products within price range and in stock, independent of keyword if keyword is absent
R5	Yes	Yes	No	Yes	Include out-of-stock items only if stock filter is off; clarify UI/API default
R6	No	No	No	No	Return default/browse results or validation response as specified

Completeness and test execution

- Build a decision table from the actual requirements; merge rules with identical actions and mark impossible combinations.

- For each rule, create seed records that should be included and excluded, including null fields and exact price boundaries.
- Test AND/OR/NOT explicitly: a product matching only one of two OR conditions should appear; a product matching one AND condition only should not.
- Test contradictory filters, empty lists, repeated parameters, unknown category, min price greater than max price, and missing/null values.
- Verify count, sorting, pagination, duplicate elimination, and query parameterization; inspect generated SQL to check that user input cannot alter query structure.

The example table is illustrative, not a replacement for the application's actual business rules. A complete decision table must cover every meaningful condition combination or use justified rule reduction. Pairwise testing may reduce effort, but it should not replace full coverage for high-risk filter interactions.

Q11. Smart Thermostat: Nested Loops and Calibration

A trace with recurring nested loops suggests repeated sampling and adjustment, but loop invariants and termination rules must be confirmed from trace values, firmware specifications, and controlled experiments. Typical routines may include sensor sampling, averaging/filtering, comparison with a target, actuator adjustment, and validation of stability.

Logical sub-routines

- Read sensor: obtain temperature and verify the reading is in a plausible range.
- Filter/calibrate: collect a fixed number of samples and calculate an average or correction.
- Adjust control value: compare measured and target temperature and apply a bounded correction.
- Check convergence: determine whether error is within tolerance for the required consecutive samples.
- Timeout and fail-safe: stop calibration after a maximum iteration/time budget and enter a safe state if readings remain invalid.

Loop invariants and termination

Possible invariants include: the sample counter never exceeds its configured maximum; the accumulated sum equals the sum of samples already accepted; the correction remains within safe bounds; and every iteration either progresses toward termination or consumes

a finite retry budget. The outer loop may repeat calibration rounds while error exceeds tolerance; the inner loop may collect a fixed number of samples.

Boundary-focused tests

Test	Condition	Expected behavior
TH-01	Zero samples configured or missing configuration	Reject configuration or use safe documented default; no infinite loop
TH-02	One sample	Terminates after one valid sample
TH-03	Sample count at maximum and maximum+1	Accept maximum; reject/clamp excess per specification
TH-04	Error just outside, exactly at, and just inside tolerance	Correct convergence decision
TH-05	Sensor returns invalid/out-of-range values	Discard or fault according to policy; bounded retries
TH-06	Sensor never stabilizes	Maximum-iteration or time limit triggers safe failure
TH-07	Counter or accumulator near numeric limit	No overflow; valid bounded result
TH-08	Nested-loop worst-case configuration	Meets timing deadline and watchdog requirements

Path testing should cover zero/one/many iterations, normal convergence, early termination, invalid sensor readings, and timeout. Static analysis, watchdog tests, and hardware-in-the-loop testing add confidence that the firmware will not hang or drive actuators unsafely.

Q12. Tax Calculation DLL: ECP and BVA for Hidden Slabs

The DLL's output codes and calculated results can be used to infer income brackets, but the test engineer must distinguish a tax slab from an error code. Error codes may represent invalid input, unsupported residency/status, or internal failures rather than tax categories. Confirm the jurisdiction, tax year, filing status, deductions, and rounding rules before asserting legal correctness.

Functional components

- Input parser: accepts a numeric income and required taxpayer attributes.

- Validation: rejects negative income, malformed values, unsupported status, and values outside documented numeric capacity.
- Slab selection: assigns income to the appropriate bracket.
- Tax computation: applies marginal rates, deductions, credits, and exemptions in the correct order.
- Output mapping: returns tax amount, effective rate, status, and documented error code.

ECP and BVA design

Input class	**Test values / checks**
Invalid income	Blank, non-numeric, negative, NaN/infinity if accepted by interface; expect controlled validation error
Each slab boundary B	B−smallest currency unit, B, B+smallest currency unit; verify bracket and marginal calculation
Zero and low income	0, minimum positive amount, below exemption threshold, exact exemption threshold
Upper range	Maximum supported income, maximum+one unit, very large integer; no overflow or silent wraparound
Other tax variables	Each filing status, residency, deduction, or credit class with valid and invalid values
Output code	Known valid result, each documented error, unknown/internal failure; stable mapping and no ambiguous output

Validation and limitations

For a progressive tax system, crossing a bracket boundary normally changes only the portion taxed at the higher marginal rate, not the rate applied to all income. Test expected results using an independently implemented calculation or authoritative rules for the relevant year. Verify rounding, currency precision, deductions, and output consistency. The reconstructed suite validates the DLL against an agreed tax specification; it cannot prove compliance with “hidden tax laws” unless the applicable law and year are known.

Q13. Flight Booking: Cause-Effect Graphing and Path Testing

The defect occurs only when Frequent Flyer status, International travel, and Last-minute booking are combined. This indicates an interaction among conditions that isolated tests or simple pairwise tests may have missed. Possible causes include a discount/fee precedence issue, an eligibility rule, an international surcharge, or a branch that is reached only for urgent bookings.

Cause-effect model

Cause	Example effect / rule to verify
C1: Frequent Flyer = Yes	May qualify for loyalty benefits or priority handling
C2: Travel = International	May require passport/visa validation, international fees, and currency rules
C3: Booking = Last-minute	May trigger urgency surcharge, restricted fare inventory, or cutoff logic
E1: Price calculation	Correct combination of loyalty discount and last-minute/international charges
E2: Eligibility/validation	Correct checks for travel documents, loyalty eligibility, and booking cutoff
E3: Confirmation	One consistent reservation and payment state; no duplicate booking on retry

Hybrid testing strategy

- Create a cause-effect graph and derive a decision table for all meaningful combinations of the three causes, plus related conditions such as loyalty tier, fare class, time to departure, and payment status.
- Test all eight combinations of the three Boolean conditions as a baseline, then add boundary values around the last-minute cutoff and international-route rules.
- Construct CFG paths through eligibility checks, price calculation, document validation, payment authorization, seat reservation, and confirmation.
- Exercise success, validation failure, payment decline, timeout, seat inventory change, and rollback/retry paths.
- Use MC/DC or condition coverage for compound Boolean expressions and add regression tests for the exact three-way combination that exposed the defect.
- Assert both displayed and charged price, discount/surcharge breakdown, reservation state, and audit trail.

Why the hybrid approach is stronger

Cause-effect graphing identifies combinations of business conditions that should produce distinct outcomes. Path testing verifies that the internal routes implementing those

outcomes are exercised. Together they address both missing requirement combinations and untested code branches. “Maximum defect detection” cannot be guaranteed, but risk-based combination coverage, boundary tests, and branch/path metrics substantially improve confidence.

Overall Conclusion

Reverse engineering test design requires evidence-based inference rather than assuming that logs reveal the complete implementation. For black-box systems, use equivalence partitioning, boundary value analysis, decision tables, and state-transition testing. For code or traces, build a CFG and measure statement, branch, condition, and basis-path coverage. For integrations and resource-sensitive modules, add fault injection, concurrency, stress, security, and recovery tests. Record assumptions, expected outcomes, coverage gaps, and regression cases so that the reconstructed suite remains auditable.